The Globe oposiciones

Bloque 3 - Tema 6 ejercicios 1

ARQUITECTURA JAVA EE/JAKARTA EE Y PLATAFORMA .NET: COMPONENTES, PERSISTENCIA Y SEGURIDAD. CARACTERÍSTICAS, ELEMENTOS, LENGUAJES Y FUNCIONES DE AMBOS ENTORNOS. DESARROLLO DE INTERFACES

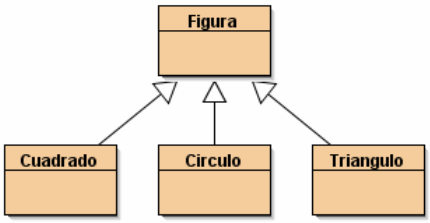
Soluciones

PREPARACIÓN OPOSICIONES

TÉCNICOS AUXILIARES DE INFORMÁTICA

Enunciado 1

Dada la siguiente jerarquía de clases:



Implementar en Java aquellos atributos y métodos necesarios para que se pueda ejecutar el siguiente programa teniendo en cuenta que deberá aparecer por pantalla el área de cada una de las figuras creadas.

```
public class Jerarquia {
      public static void main(String[] args) {
            Vector<Figura> figuras = new Vector<Figura>();
            figuras.add(new Circulo(10)); // Radio=10
            figuras.add(new Cuadrado(10)); // Lado=10
            figuras.add(new Triangulo(10,5)); // Base=10, Altura=5;
            for (Figura f: figuras)
                  System.out.println("Área: "+f.area());
            }
      }
}
```

SOLUCIÓN

```
public abstract class Figura {

    public abstract double area();
}

public class Cuadrado extends Figura{
    private double lado;

    public Cuadrado() { lado = 0; }

    public Cuadrado(double lado) {
        this.lado = lado;
    }

    public double area() {
        return lado * lado;
    }
}

public class Circulo extends Figura {
    private double radio;

    public Circulo(){}
```

```
    public Circulo(double radio) {
        this.radio = radio;
    }

    public double area() {
        return Math.PI * radio * radio;
    }
}

public class Triangulo extends Figura {
    private double base;
    private double altura;

    public Triangulo() {}

    public Triangulo(double base, double altura) {
        this.base = base;
        this.altura = altura;
    }

    public double area() {
        return base * altura / 2;
    }
}

import java.util.Vector;

public class Jerarquia {
    public static void main(String[] args) {
        Vector<Figura> figuras = new Vector<Figura>();
        figuras.add(new Circulo(10)); // Radio=10
        figuras.add(new Cuadrado(10)); // Lado=10
        figuras.add(new Triangulo(10,5)); // Base=10, Altura=5;

        for (Figura f: figuras)
            System.out.println("Área: "+f.area());
    }
}
```

Enunciado 2

Dado el siguiente código:

```
public class Electrodomestico {
	private double consumo;
	private String modelo;

	public Electrodomestico (double c, String m) {
		consumo = c;
		modelo = m;
	}

	public String toString () {
		return "Modelo: " + modelo + " Consumo: " + consumo;
	}
}

public interface MuestraInformacion {
	public String muestra();
}
```

Se pide desarrollar la clase Frigorifico:

```
public class Frigorifico extends Electrodomestico
                implements MuestraInformacion {
	double temperatura;

	public Frigorifico (double c, String m, double t) { ... }
	public String toString () { ... }
	public String muestra () { ... }
}
```

Teniendo en cuenta que es necesario:

- **Implementar el constructor de Frigorifico.**
- **Implementar el método toString de Frigorifico.**
- **Implementar el método muestra de Frigorifico para que muestre su temperatura.**

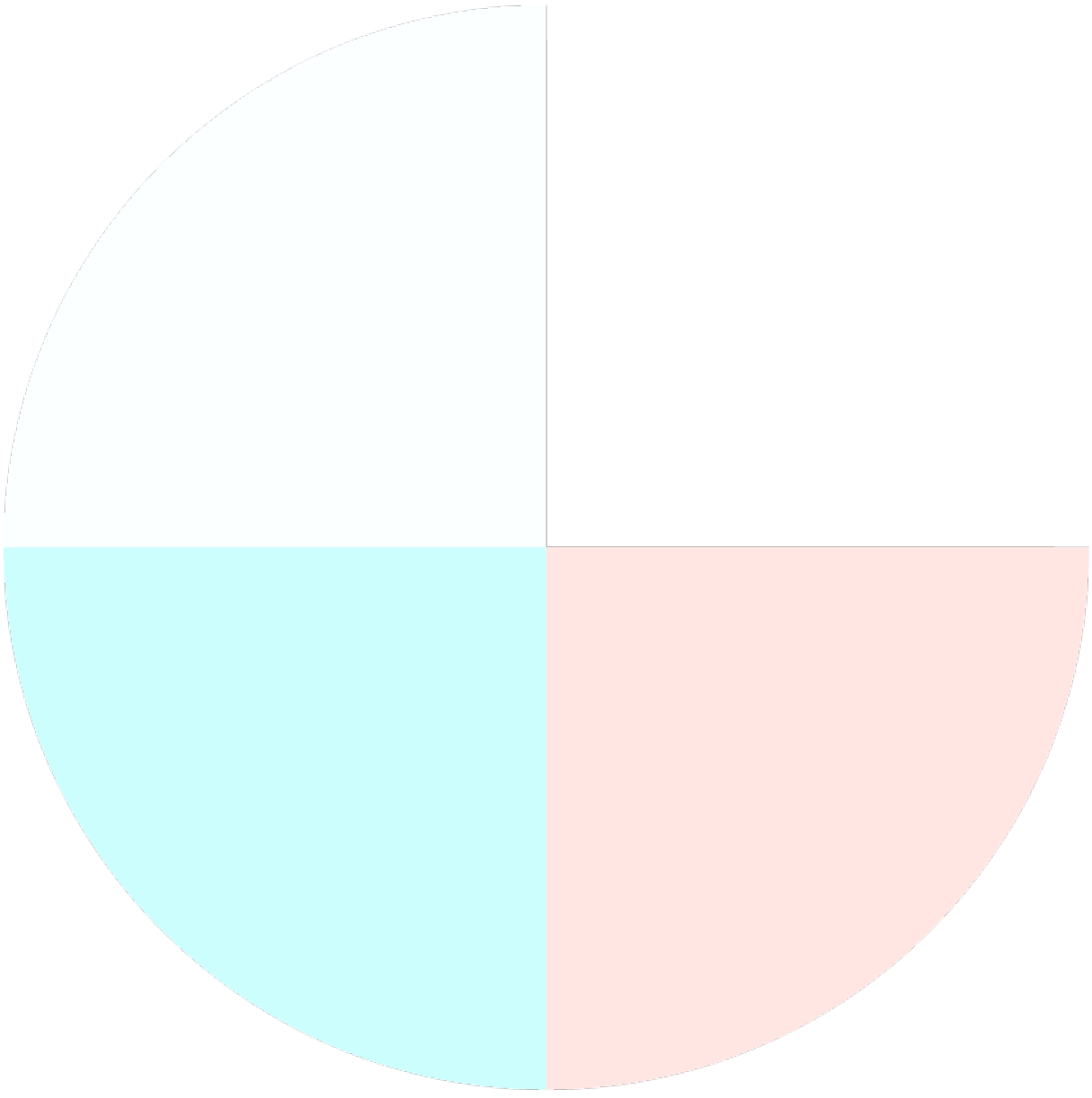
SOLUCIÓN

```
public class Frigorifico extends Electrodomestico
        implements MuestraInformacion {
    double temperatura;

    public Frigorifico (double c, String m, double t) {
        super(c,m);
        temperatura = t;
    }
```

```
    public String toString() {
        return super.toString() + "Temperatura: " + temperatura;
    }

    @Override
    public String muestra() { return "Temperatura: " + temperatura; }
}
```



Enunciado 3

Implemente las siguientes clases en Java:

a) **Clase Nota.** Una nota contiene un identificador numérico y una línea de texto. Defina constructores, getters y setter y toString.

b) **Clase NotaAlarma.** Una nota que además contiene la hora en la que sonará la alarma. Defina constructores, getters y setter y toString.

c) **Clase BlocNotas.** Modela un bloc de notas en el que se pueden introducir notas, listar todas las notas, eliminar una nota mediante su posición en el bloc de notas o saber cuántas notas contiene el bloc de notas. Debe utilizar la clase Array.

d) **Clase Prueba.** Cree un bloc de Notas de ejemplo y pruebe las operaciones que soporta.

SOLUCIÓN

```
public class Nota {
    private int idNota;
    private String linea;

    public Nota() {
    }

    public Nota(int idNota, String linea) {
        this.idNota = idNota;
        this.linea = linea;
    }

    public int getIdNota() {
        return idNota;
    }

    public String getLinea() {
        return linea;
    }

    public void setIdNota(int idNota) {
        this.idNota = idNota;
    }

    public void setLinea(String linea) {
        this.linea = linea;
    }

    @Override
    public String toString() {
        return "IdNota: " + idNota + " Linea: " + linea;
    }
}
```

```
public class NotaAlarma extends Nota {
    private String hora;

    public NotaAlarma(String hora) {
        this.hora = hora;
    }

    public NotaAlarma(int idNota, String linea, String hora) {
        super(idNota, linea);
        this.hora = hora;
    }

    public String getHora() {
        return hora;
    }

    public void setHora(String hora) {
        this.hora = hora;
    }

    @Override
    public String toString() {
        return super.toString() + "Hora=" + hora;
    }
}


import java.util.ArrayList;

public class BlocNotas {
    private ArrayList<Nota> notas = new ArrayList<>();

    public BlocNotas() {}

    public void add(Nota n) {notas.add(n);}

    public int numeroNotas() {return notas.size();}

    public ArrayList<Nota> list() {return notas;}

    public void remove(int pos) {notas.remove(pos);}
}

public class PruebaBloc {

    public static void main(String[] args) {
        BlocNotas bloc = new BlocNotas();

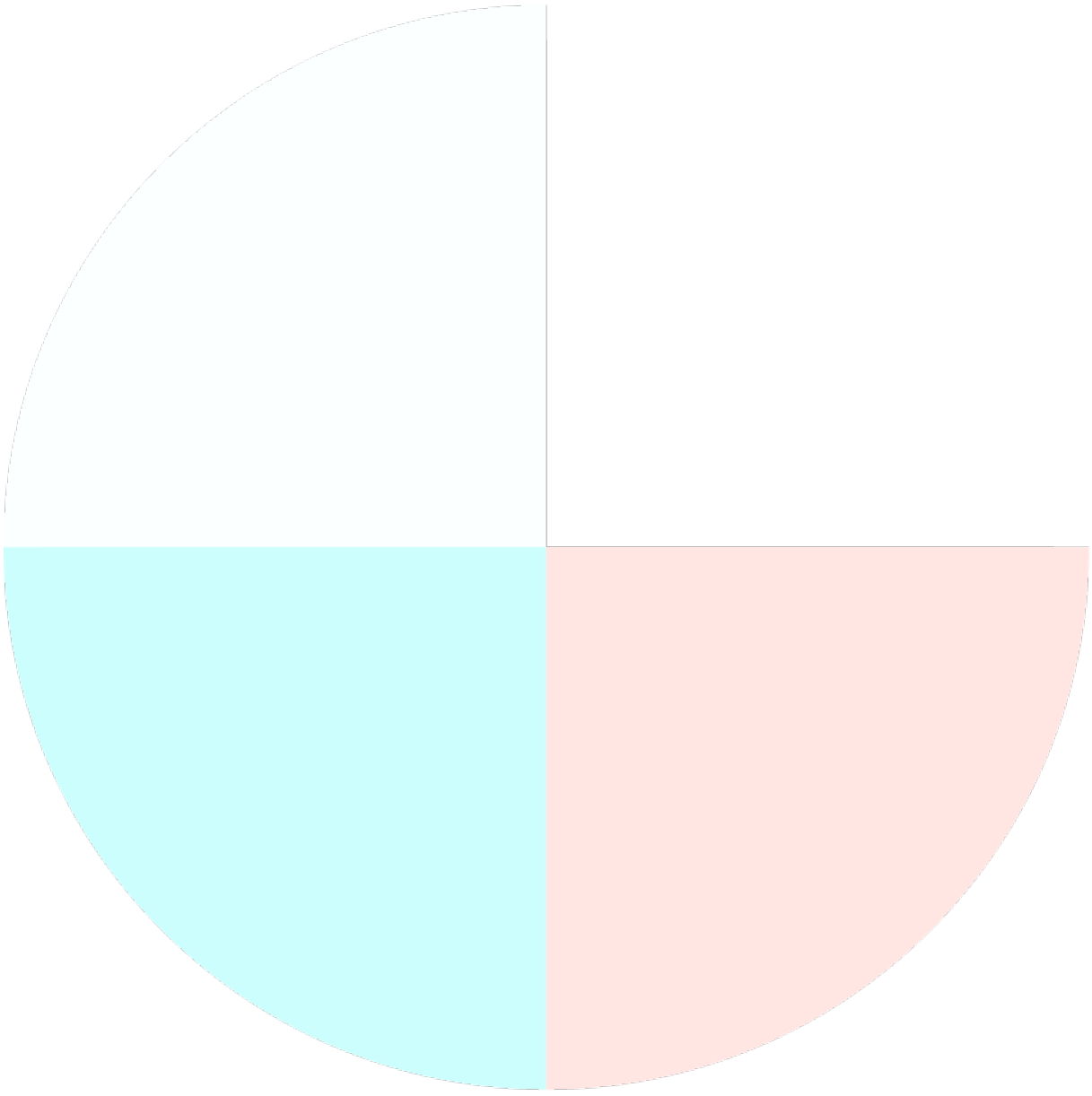
        bloc.add(new Nota(1,"Linea 1 del bloc"));
        bloc.add(new NotaAlarma(2,"Nota de alarma","16:30"));
        bloc.add(new Nota(3,"Mi nota 3"));

        System.out.println("Numero de notas del bloc:" +
bloc.numeroNotas());

        System.out.println("Listado notas del bloc: " + bloc.list());

        bloc.remove(2);
```

```
        System.out.println("Listado notas del bloc: " + bloc.list());
    }
}
```



Enunciado 4

Explique detalladamente qué hace el siguiente código:

```
import java.sql.*;

public class Acceso {
      private static Connection conexión = null;

      public static void main(String[] args) {
            try {
                  conexión =
                        DriverManager.getConnection("jdbc:racle:thin:@10.
                        100.23.22:1524:equipo1", "opencms", "opencms");

            } catch (SQLException e) {
                  e.printStackTrace();
            }
            System.out.println("Conexión creada con éxito…");
            try {
                  Statement stmt = conexión.createStatement();
                  ResultSet rs = stmt.executeQuery("SELECT IP, UNIDAD,
                              PROVINCIA FROM MAPA");
                  while(rs.next()) {
                        System.out.println("dirección IP Organismo
                              Provincia");
                        System.out.println(rs.getObject(1) + " " +
                              rs.getObject(2) + " " + rs.geObject(3));
                  }
            } catch (SQLException exc) {
                  System.out.println("Error en ejecutar Consulta " +
                        exc.getMessage());
            }
      }
}
```

SOLUCIÓN

Se ha implementado la clase Acceso. Tiene un único método, el método main. Esta clase hará uso de las clases del paquete java.sql.

Traza de la ejecución del método main:

(1) En un bloque try se intenta conectar con la base de datos Oracle con las credenciales opencms/opencms.

(2) Si en el intento de conexión se produce una excepción del tipo SQLException, será tratado por el único bloque catch que muestra la traza de error por pantalla.

(3) En el siguiente bloque try:

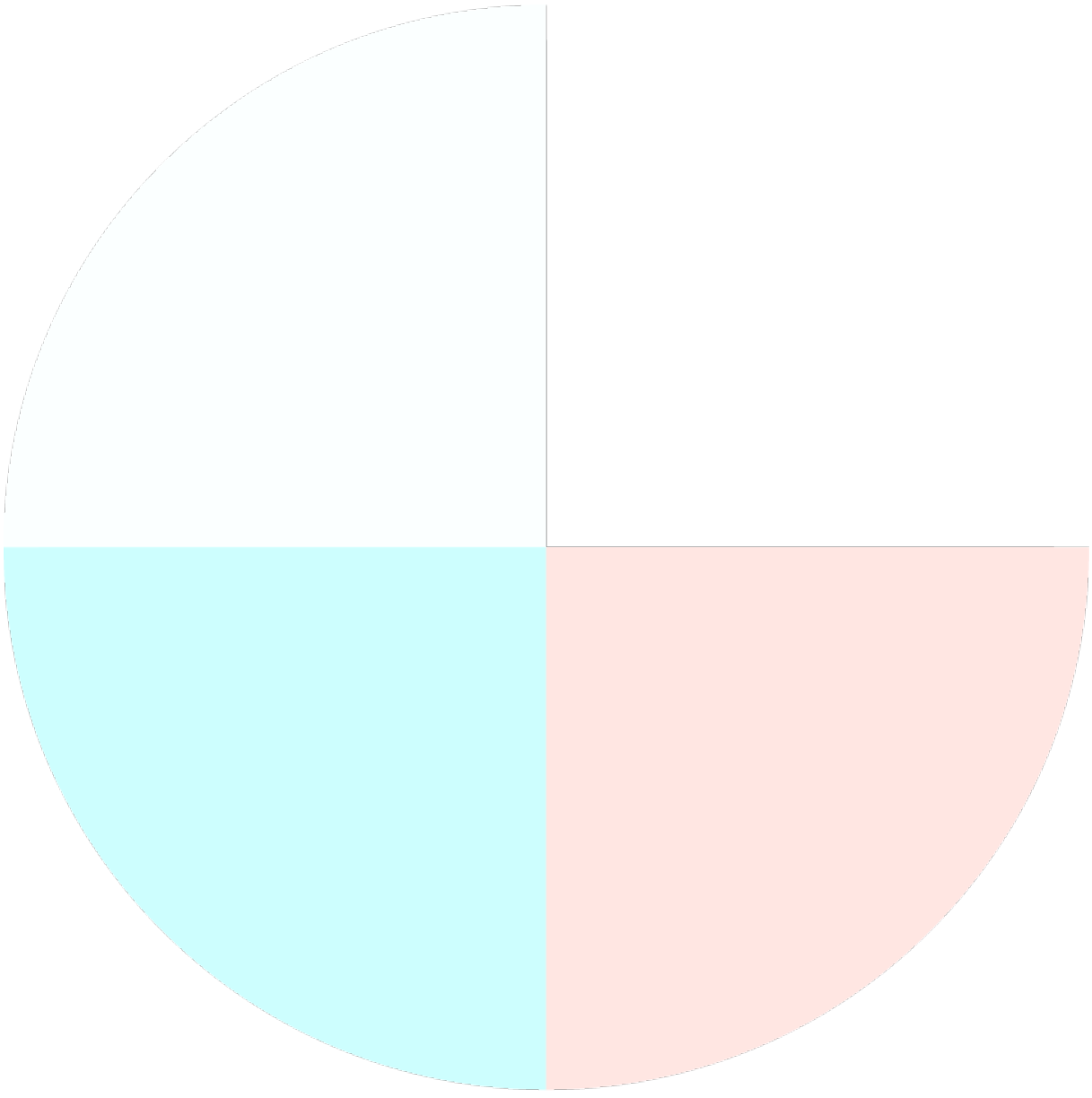
1º. Se crea una sentencia stmt.

2º. Se ejecuta la consulta sobre la base de datos y el resultado se guarda en el objeto rs.

3º. En un bucle while se comprueba si rs tiene datos y mientras los tenga se ejecuta lo siguiente:

1. Se muestra por pantalla la IP, el organismo y la provincia.

(4) Si al ejecutar la consulta se produce una excepción del tipo SQLException será tratada por el único bloque catch que muestra por pantalla el error devuelto al intentar consultar la base de datos.



Enunciado 5

Explique detalladamente qué hace y la salida que imprime por pantalla el siguiente código:

```
public class Empleado {
      void imprimir() {System.out.println("Empleado");} }

public class Interino extends Empleado {
      void imprimir() {System.out.println("Interino");} }

public class Recursos {
   public static void main(String[] args) {
         Empleado emp = new Empleado();
         emp.imprimir();
         Interino inter = new Interino();
         inter.imprimir();
         Empleado e = new Interino();
         e.imprimir();
   }
}
```

SOLUCIÓN

Se han implementado tres clases:

- Empleado: tiene un método imprimir que muestra por pantalla la cadena Empleado. El constructor (implícito) es el constructor por defecto.
- Interino: clase que extiende de Empleado. Sobreescribe/redefine el método imprimir para que muestre por pantalla la cadena Interino.
- Recursos: clase que contiene el método main, por donde comenzará la ejecución.

El concepto clave es la ligadura dinámica, es decir, en tiempo de ejecución se comprueba para el objeto la clase sobre la que ha sido instanciado. No se conoce en tiempo de compilación.

Traza de la ejecución del método main:

(1) Se declara el objeto emp del tipo Empleado y se instancia como un objeto de Empleado.

(2) Se llama al método imprimir de Empleado pues emp es un objeto instanciado de esa clase.

(3) Se declara el objeto inter del tipo Interino y se instancia como un objeto de Interino.

(4) Se llama al método imprimir de Interino pues inter es un objeto instanciado de esa clase.

(5) Se declara el objeto e del tipo Empleado y se instancia como un objeto de Interino.

(6) Se llama al método imprimir de Interino pues e es un objeto instanciado de Interino.

Por tanto, la salida por pantalla es:

Empleado
Interino
Interino